Software engineering: est l'application systématique d'approches d'ingénierie au développement de logiciels.

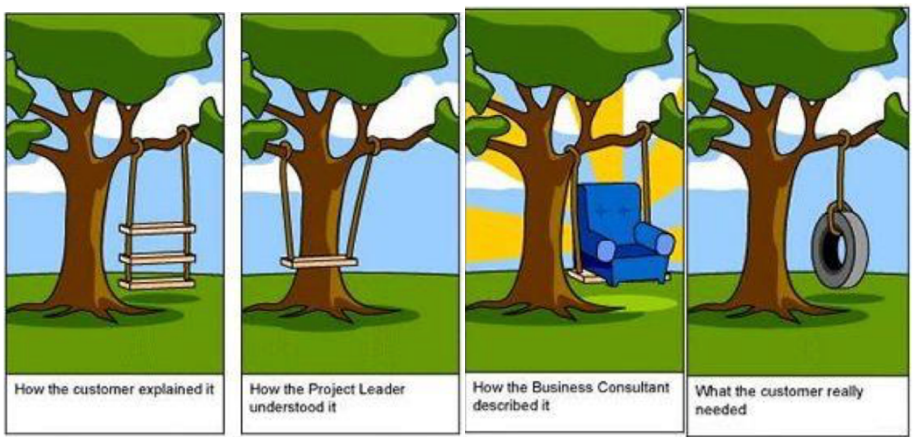
Software development: est le processus de conception, de spécification, de conception, de programmation, de documentation, de test et de correction de bogues impliqué dans la création et la maintenance d'applications, de frameworks ou d'autres composants logiciels.

Software development Life Cycle (SDLC) : Le cycle de vie du développement logiciel est un processus qui produit des logiciels de la plus haute qualité et au coût le plus bas dans les plus brefs délais.

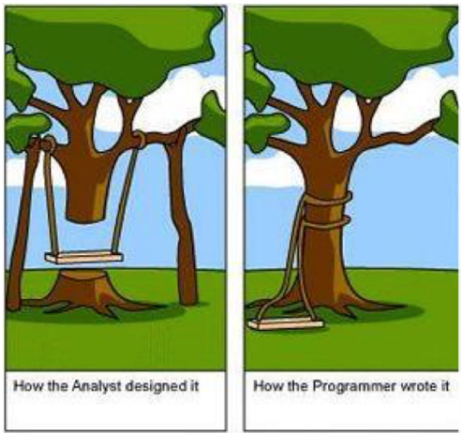
- **Requirements:** détermine le besoin du cliente
- **Design:** définir les spécifications et la structure du logiciel. Cette étape est parfois divisée en conception de haut niveau (architecture) et en conception détaillée.
- **Implementation:** création proprement dite du logiciel (codage, configuration)
- **Testing:** évaluer si le logiciel répond aux spécifications et aux attentes des clients
- **Maintenance:** une fois en production, améliorer le logiciel et s'assurer qu'il reste opérationnel

Expliquons ça avec photos :

1) **Requirement***:* détermine le besoin du cliente**,** si on vend trop de rêve à la fin le cliente va être déçu et fâché donc c'est très important de ne pas exagéré et de ne pas mentir au cliente .

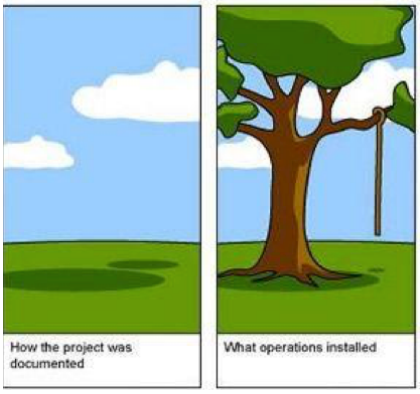


2) **Design and Implementation :**

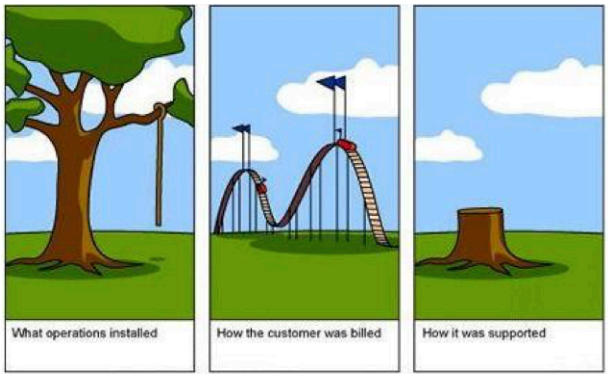


3) **Testing*:*** Dans cette étape, nous testons les défauts et les carences. Nous résolvons ces problèmes jusqu'à ce que le produit réponde aux spécifications d'origine. En bref, nous voulons vérifier si le code répond aux exigences définies.

   Il est important de garder en tête si on a quelque chose qui marche sur notre laptop ça veut pas dire que va marcher pour cliente.



4) **Maintenance and operation:** once in production, improve the software and make sure it remains operational



On a 2 type de besoins(requirement) :

- **Fonctional** : les besoins fonctionnel vont être validé par l'utilisateur, c.à.d est-ce que j'ai fait ce que l'utilisateur voulais. C'est relativement difficile à valider.

- **Non-functional** : les besoins fonctionnel non fonctionnel la façon dont le logiciel va se comporter. Est-ce qu'il est performent? est-ce qu'il est sécurisé ? est-ce qu'il facile à maintenir ? est-ce qu'il peut opérer facilement avec des autres intervenants ?

Comment évaluer la qualité de requirement?

- **unitaire** : les besoins adresse uniquement à un objective. Si les besoins adresse à plusieurs objectives très probablement il faut le casser.
- **Completeness** : vérifier que les besoins sont complet c.à.d il fait sens en tant que telle.
- **Consistance :** un besoin ne viole pas les autres exigences
- **Traceable** : Il est documenté .et tous a bien compris.
- **Current :** les exigences sont à jour.
- **Sans ambiguïté:** les exigences sont suffisamment documentées pour ne pas exiger de connaissances supplémentaires pour être interprétées
- **Priorité:** les exigences sont correctement hiérarchisées pour aider à optimiser les ressources
- **Vérifiable:** la mise en œuvre de l'exigence peut être vérifiée et il existe un moyen clair de valider si elle a été correctement mise en œuvre

Comment capturer requirement?

1. Haut niveau / Pitching:
    a) Document vision
    b) Lean canvas
2. Détaillée
    a) USER STORIES / USE CASES
    b) STORYBOARDS
    c) NON- FUNCTIONAL

1.a) Document vision :

Établir une **vision commune du projet,** c'est la déclaration initiale (et souvent fondamentale) de votre projet. Il fournit une plate-forme pour communiquer (et collaborer sur) la forme et la portée de votre projet avec d'autres partenaires principaux du projet et des collaborateurs externes potentiels. Un document vision de projet ne doit jamais dépasser 2 pages.

Titre: le nom de votre projet?

Description du projet: Décrivez le projet. Quelles sont ses motivations et sa signification intellectuelle. Pourquoi c'est important?À quel type de besoin répond-il?

Produits livrables / résultats: Quel sera le résultat final, que produira le projet. Si cela aboutit à un site Web, discutez de ce site Web.

Fonctionnalité et technologie : Décrivez la fonctionnalité du projet. Quels types de technologie seront utilisés pour construire tous les aspects du projet.

Public: Quel est le public (et pourquoi)? Pensez à comment allez-vous créer une communauté parmi votre public?

Données, informations, contenu : Quel est le contenu / les données de base du ? d'où proviennent-ils? N'oubliez pas que si vous tirez des données / contenus d'une autre institution (musée, bibliothèque numérique, archives, etc.), vous devriez envisager de les inclure comme partenaires.

Les partenaires : Votre projet a-t-il des partenaires (institutionnels)? Si c'est le cas, discutez brièvement de qui ils sont et quel est leur rôle dans le projet.

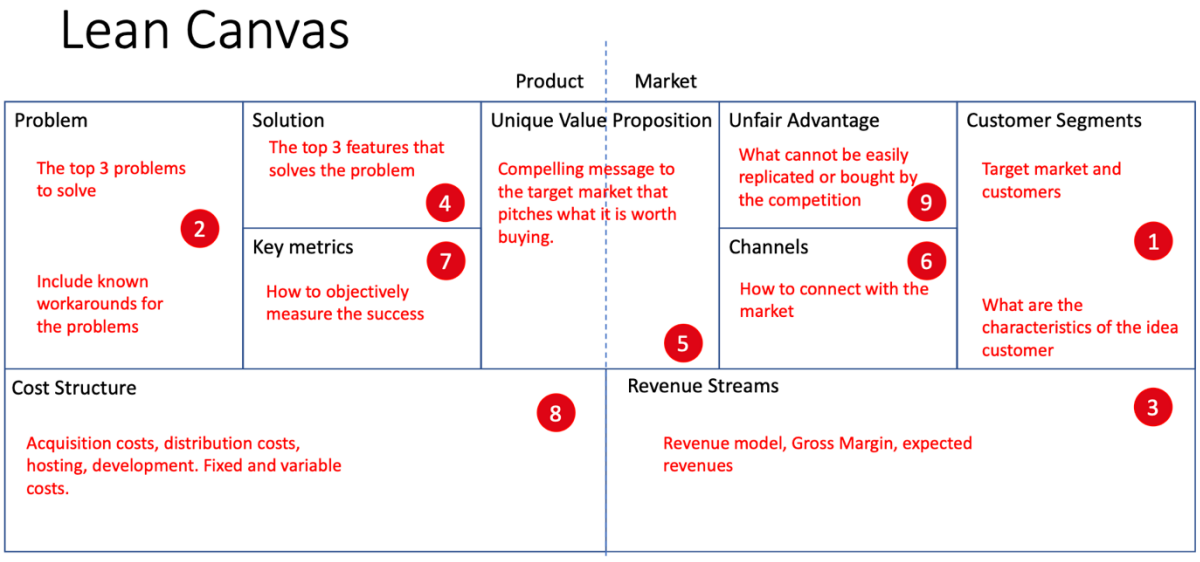
1.b) Lean Canvas :

Lean Canvas présente l'hypothèse d'un business model sous la forme d'un document d'une page.

Le Lean Canvas rassemble toutes ces informations dans un format hautement portable et facilement lisible. Lean Canvas est une approche problème-solution. Cela aide à éliminer tout ce qui est inutile dès le début. Il économise du temps et des ressources et peut être mis à jour en permanence. L'objectif principal de Lean Canvas est **d'augmenter le taux de réussite de l'entreprise,** ainsi que de **réduire le risque d'échec**.

Lean Canvas permet de répondre aux questions de la plupart des startups lean qui souhaitent devenir de vraies entreprises:

- Quel est notre produit?
- Qui sont nos clients?
- Comment pouvons-nous gagner de l'argent?
-



Un Lean Canvas se compose de 9 blocs que vous devez remplir pour réfléchir à vos modèles commerciaux possibles.

Problème: Tout d'abord, vous devez définir un problème, puis y trouver une solution. Dans cet encadré, décrivez brièvement trois problèmes prioritaires qui doivent être résolus.

Customer Segment(CS): Quelles sont les personnes qui ressentent la douleur mentionnée ci-dessus? Où vivent-ils? Où travaillent-ils? Quels produits utilisent-ils? Comment pouvez-vous rendre leur vie plus facile et meilleure? Affinez les caractéristiques essentielles de chacun de vos segments de clientèle. Définissez vos premiers utilisateurs - ils sont votre premier marché. Au stade initial, identifiez un seul type de client. et si vous avez plusieurs groupes (comme des développeurs de logiciels, des comptables), vous feriez mieux de créer un Lean Canvas distinct pour chacun d'entre eux. Les zones Problème et Segments de clients sont interconnectées.

Solution : De quelle manière votre produit résoudra-t-il les problèmes de CS ? Quel ensemble de services pouvez-vous proposer pour répondre aux besoins spécifiques de CS? Une fois que vous avez compris la douleur et posé un diagnostic, vous pouvez prescrire le médicament.

Channels :
Découvrez comment vous allez atteindre votre CS et proposez-lui votre produit. Gardez à l'esprit qu'il n'y a pas du tout de chaînes gratuites. Quel que soit l'itinéraire que vous choisissez, vous devrez investir de l'argent et du temps. Un premier canal idéal est le marketing de contenu, car avec le temps, il se transforme en un actif tangible.

Revenue Stream:
Réfléchissez à la stratégie de prix de votre produit. Expliquez comment vous allez gagner de l'argent et rédigez un modèle de monétisation.

Cost Structure:
Dressez la liste de tous les coûts opérationnels liés à la commercialisation de votre produit. Combien devez-vous dépenser en salaires? Combien coûte votre site Web?

Key metrics :
Ce que vous ne pouvez pas mesurer, vous ne pouvez pas contrôler. Certainement, vous devez surveiller vos performances. choisissez quelques métriques macro clés qui comptent et concentrez-vous sur celles-ci. Identifier les bonnes mesures ici est crucial car des mauvais choix peut conduire à poursuivre les mauvais objectifs, une recette infaillible pour un échec catastrophique. Assurez-vous que vos indicateurs sont fermement liés à la réalité.

Unfair Advantage :
Pensez à ce que vous pouvez offrir à vos clients que personne d'autre ne peut, et ce sera votre défense active contre les copieurs. Le premier jour, vous pouvez laisser cette boîte vide, mais ne l'oubliez pas.

2.a) USER STORIES / USE CASES

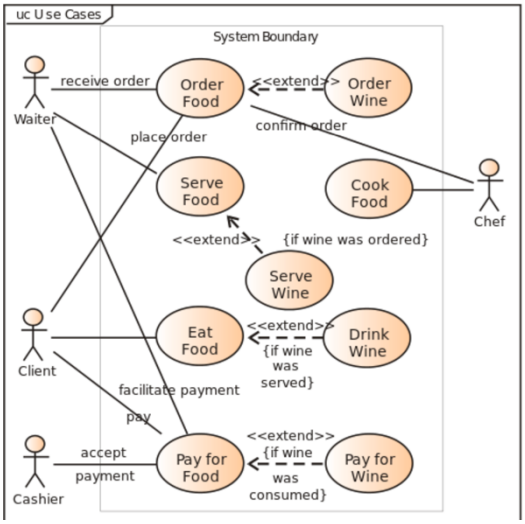
USER STORIES:
Les user stories sont des descriptions courtes et simples d'une fonctionnalité racontée du point de vue de la personne qui souhaite la nouvelle fonctionnalité, généralement un utilisateur ou un client du système. Ils suivent généralement un modèle simple:

As a < type of user >, I want < some goal > so that < some reason >

Exemple : En tant que visiteur dans un pays étranger, je souhaite pouvoir réserver un taxi en un clic en connaissant le coût à l'avance pour ne pas me faire foutre car je ne connais pas les coutumes locales.

Use Case : est une description textuelle de la fonctionnalité fournie par le système.
Décrit les interactions entre un rôle alias un acteur et le système pour atteindre un objectif particulier. . Il capture l'interaction acteur-système. Il spécifie comment un utilisateur interagit avec un système et comment le système répond aux actions de l'utilisateur. Il est

souvent formulé sous la forme d'un dialogue entre l'acteur et le système. La spécification de Use case est représentée dans le diagramme de use case par un ovale.

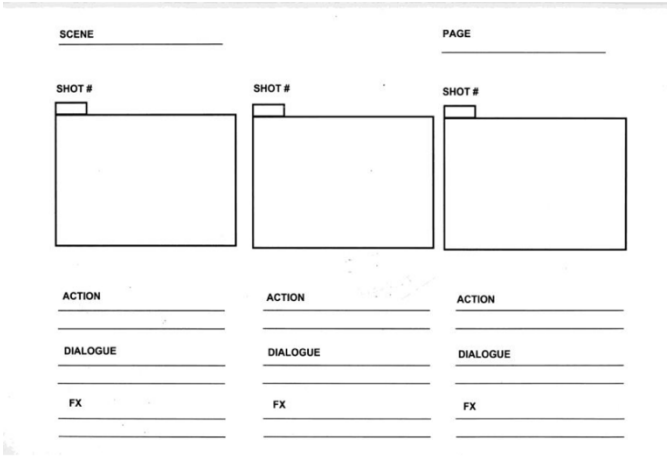


Use case VS user stories :

Ma réponse standard est que les user stories sont centrées sur le résultat et l'avantage de ce que vous décrivez, alors que les use case sont plus granulaires et décrivent comment votre système va agir.

2.b) STORYBOARDS

Racontez une histoire avec une séquence de dessins / croquis / images



2.c) NON- FUNCTIONAL

définissent comment le système fonctionne PAS ce qu'il fait. Il décrit les capacités de fonctionnement du système et les contraintes qui améliorent sa fonctionnalité. Il peut s'agir de vitesse, de sécurité, de fiabilité, etc.

Non-Functional requirement Aussi appelés attributs de qualité:
1. Attributs de qualité d'exécution : sécurité, convivialité, performances (débit, temps de réponse), tolérance aux pannes, haute disponibilité, auditabilité .... Disponibilité

2. Attributs de qualité d'évolution : Maintenabilité, testabilité, interopérabilité, performances (évolutivité)